

Tour Ceylon — Itinerary Generator

Demonstration & Explanation Guide
From user login to day-by-day plan · 15 August 2026

1. What you are demonstrating

The **Travel Itinerary Generator** is part of Tour Ceylon. A tourist logs in, picks attractions and budget, chooses hotels, then clicks Generate to receive a **day-by-day timed schedule**. The engine is **rule-based** (not ChatGPT, not Random Forest). It reads real data from PostgreSQL and applies fixed logic for day splitting and clock times.

Item	Value
Travel app URL	http://localhost:5180
Backend API	http://127.0.0.1:5002/api
Database	PostgreSQL — travel_app
Demo flow	Login → Attractions → Budget → Stay → Itinerary → Export PDF

2. User login (before the wizard)

Every trip is tied to a logged-in user. Without login, the app redirects to the login page.

File	Role
frontend/travel/src/pages/LoginPage.jsx	Login / register form (email + password, optional Google)
frontend/travel/src/context/AuthContext.jsx	Stores JWT token in localStorage (access_token)
frontend/travel/src/App.jsx	ProtectedRoute — blocks wizard until authenticated
backend/app/routes/auth.py	POST /api/auth/login and /api/auth/register
backend/app/models/user.py	users table — email, password_hash

Login flow (simple):

- User enters email + password on LoginPage.
- Browser calls POST /api/auth/login → auth.py checks users table.
- Server returns JWT access_token (valid ~7 days).
- Token saved in localStorage; all later API calls send Authorization: Bearer <token>.
- App redirects to /attractions — wizard step 1.

```
POST /api/auth/login
{ "email": "visitor@example.com", "password": "secret123" }
→ { "access_token": "<jwt>", "user": { "id": 1, "email": "..." } }
```

3. Trip wizard (data collected before itinerary)

The itinerary cannot run until steps 1–3 are complete. Each step saves data to **user_trip_input** and related tables.

Step	Page	API	What is saved
1	AttractionsPage.jsx	POST /api/trip-input	days, moods, attraction IDs
2	BudgetPage.jsx	POST /api/budget/split	total budget + food/stay/shopping/transport split
3	AccommodationPage.js x	PATCH /api/trip-input	hotel per destination + room type (Single/Double/Family)
4	ItineraryPage.jsx	POST /api/itinerary/generate	← GENERATES the plan
5	RecommendationsPage.js x	GET /api/business-directory etc.	optional saved shops/agencies/guides
6	ExportPage.jsx	GET /api/itinerary/<id>/pdf	download PDF

Backend files for steps 1–3:

- routes/trip_input.py — create/update trip
- routes/budget.py + services/budget_service.py — split budget percentages
- routes/accommodation.py + trip_plan_service.py — list hotels within budget
- models/user_trip_input.py — main trip row (days, attractions, hotels, room_type)
- models/budget_split.py — budget categories
- Data seeded from Excel: attractions.xlsx (130 places), accomadation.xlsx (944 hotels)

4. How the itinerary is generated (core component)

When the user opens Itinerary and clicks **Generate**, this chain runs:

```
ItineraryPage.jsx
  ↓ itineraryApi.generate() in client.js
  ↓ POST /api/itinerary/generate
routes/itinerary.py (validate trip, delete old plan)
  ↓ generate_itinerary(trip)
services/ai_service.py (build JSON – rules, NOT OpenAI)
  ■ uses trip_plan_service.py (days, cities, hotels)
models/generated_itinerary.py (save JSON to DB)
  ↓ response back to browser
ItineraryPage shows Day 1, Day 2, ... with times
```

4.1 routes/itinerary.py — API controller

- Checks: attractions selected, hotels selected, budget split exists.
- Loads trip with `get_user_trip()` — user can only access own trips.
- Deletes previous `generated_itinerary` rows for this trip.
- Calls `ai_service.generate_itinerary(trip, day_start)`.
- Saves result; returns JSON to frontend.

4.2 trip_plan_service.py — geography & days

- `ordered_attractions()` — load picked places from DB in user order.
- `get_day_plan()` — split trip days across cities in **consecutive blocks** (no bouncing back). Example: 7 days, 3 cities → ~2 + 2 + 3 days.
- `attractions_grouped_by_day()` — assign each attraction to the correct day number.
- `hotel_for_day()` — which hotel name applies to each day.
- Does NOT assign clock times — only structure.

4.3 ai_service.py — times & final JSON

- Despite the name, this does NOT use OpenAI or machine learning.
- Start time: user picks (e.g. 07:30 AM) OR Auto averages mood lookup table (`MOOD_START_HOURS`).
- First activity may shift by category (Wild/Heritage early, beach later) or name hints.
- `_build_day_times()` — spaces activities 2–3 hours apart, skips lunch hour, caps ~8 PM.
- Extra days in same city with no more places → 'Explore {city}' leisure block.
- Output: title, route, highlights, days[] with activities[{time, title, description}].
- Tagged source: 'database' in JSON.

4.4 generated_itinerary.py — storage

- Table: `generated_itinerary` — columns `trip_id`, `itinerary` (JSONB), `created_at`.
- Itinerary page loads this on revisit; Export page uses it for PDF.
- `pdf_service.py` reads same JSON + budget + saved references for download.

5. Worked example (what to say in demo)

Suppose user picks 7 days, 3 destinations (Colombo, Batticaloa, Hambantota), one attraction per city, Double room, budget \$700.

Stage	Result
get_day_plan()	Days 1–2 Colombo, 3–4 Batticaloa, 5–7 Hambantota
attractions_grouped_by_day()	Attidiya on day 1, Thoppigala on day 3, Bird Center on day 5
hotel_for_day()	Zest Metropole (Colombo days), Amethyst (Batticaloa), Wild Culture (Hambantota)
_parse_day_start()	Adventure mood → ~7:30 AM start if Auto selected
_build_day_times()	7:30 AM activity, gaps for lunch, no late evening slots
Saved JSON	7-Day Sri Lanka Trip + day cards shown on screen

Sample generate request:

```
POST /api/itinerary/generate
Authorization: Bearer <jwt>
{
  "trip_id": "cdb41679-....",
  "day_start": "07:30 AM"
}
→ 201 Created with full itinerary JSON
```

6. Step-by-step demonstration script

Read this aloud while clicking through the app:

Step 1 — Open <http://localhost:5180>

Show Travel app landing on login or attractions.

Step 2 — Login

Explain JWT — server verifies email, token used for all trip APIs.

Step 3 — Attractions

Pick moods, select places, set trip days. POST /api/trip-input creates trip_id.

Step 4 — Budget

Enter total budget; show pie split. POST /api/budget/split.

Step 5 — Stay

Select room type Single/Double/Family — price uses A, (A+B)/2, or B from hotel range. Pick one hotel per city.

Step 6 — Itinerary

Click Generate. Explain: server validates, trip_plan_service splits days, ai_service adds times. Show day cards.

Step 7 — Change start time

Regenerate — same places, different times. Old plan replaced in DB.

Step 8 — Discover (optional)

Save shops/agencies/guides — appear in export PDF.

Step 9 — Export

Download PDF — budget table, stays, schedule, saved references.

7. Likely panel questions & short answers

Question	Answer
Is this AI / machine learning?	No. Rule-based engine with lookup tables. OPENAI_API_KEY can stay empty.
Random Forest?	Not used anywhere in itinerary generation.
Where does attraction data come from?	Real Excel seed: attractions.xlsx → PostgreSQL.
How are days assigned to cities?	Consecutive blocks; days split evenly across destinations, extra days at later stops.
How are times chosen?	Mood average or user override; category/name hints; fixed gaps between activities.
Why can't I generate without hotels?	itinerary.py requires hotels + budget — design choice for complete plans.
Is GPS / routing used?	No. Route text is city names only; no lat/long on attractions.
How is plan persisted?	JSON in generated_itinerary table; PDF reads same data.

8. Key file checklist (print for reference)

File path	One-line job
frontend/travel/src/pages/LoginPage.jsx	User authentication UI
frontend/travel/src/context/TripContext.jsx	Wizard state in browser
frontend/travel/src/pages/ItineraryPage.jsx	Generate button + display plan
frontend/travel/src/api/client.js	HTTP calls to backend
backend/app/routes/auth.py	Login/register → JWT
backend/app/routes/trip_input.py	Create trip, save hotels
backend/app/routes/budget.py	Budget split
backend/app/routes/accommodation.py	Hotel list filtered by budget + room type
backend/app/routes/itinerary.py	Generate / get / PDF endpoints
backend/app/services/trip_plan_service.py	Days ↔ cities ↔ attractions ↔ hotels
backend/app/services/ai_service.py	Clock times + final itinerary JSON
backend/app/services/pdf_service.py	Export PDF (ReportLab)
backend/app/models/generated_itinerary.py	Saved plan storage

Tour Ceylon · Travel component · Backend port 5002 · Frontend port 5180